





Relational Model and Relational Database Schemas

From Tables to Relationships: Structuring Data Smartly



Learning Outcomes

- By the end of our session, we will be able to:
 - Explain the Relational Model and its purpose.
 - Define relation, tuple, attribute, domain, degree, and cardinality.
 - Explain why relational databases have a mathematical foundation.
 - Understand relation schema and database schema.
 - Explain super, candidate, primary, alternate, composite, and foreign keys.
 - Understand the basic idea of functional dependencies and normalization.
 - Explain NULL values and major integrity constraints.
 - Understand how SQL operations relate to relational algebra.
 - Explain why the relational model remains important alongside NoSQL.



What Is the Relational Model?

Definition

The Relational Model is a logical data model in which data is represented as relations (tables) consisting of tuples (rows) and attributes (columns).

Origin

- Proposed by Dr. Edgar F. Codd in 1970.
- Based on ideas from set theory, mathematical logic, and relational algebra.

Why was it revolutionary?

- Before the relational model:
- Hierarchical databases → rigid tree-like structures
- Network databases → powerful but complex

The relational model introduced:

- Simplicity
- Data independence
- Formal mathematical foundation
- Declarative querying
- Easier data integration

Examples:

MySQL • PostgreSQL • Oracle Database • Microsoft SQL Server • IBM Db2

- “Relational” is not simply another word for “tables.” The word comes from the mathematical concept of a relation.
- The relational model describes data at a logical level, while the DBMS handles physical storage and execution.



haribansh@bhu.ac.in

Why Is the Relational Model Mathematical?

The key idea

A relational database is called mathematical because its foundation comes from:

Set Theory + Mathematical Logic + Relational Algebra

Think of it this way:

Table = Relation = Set of Tuples

Example: Student Table:

Conceptually:

Student = { (1, Alice, 20), (2, Bob, 25), (3, Carol, 22) }

Each row → Tuple

Each column → Attribute

Complete collection of tuples → Relation



ID	Name	Age
1	Alice	20
2	Bob	25
3	Carol	22

Relations come from Set Theory

Suppose we have domains:

- $ID = \{1, 2, 3\}$
- $Name = \{Alice, Bob, Carol\}$
- $Age = \{20, 25, 22\}$

A relation can be viewed as a subset of the Cartesian product:

- $Student \subseteq ID \times Name \times Age$
- The Cartesian product:
- $ID \times Name \times Age$ represents all theoretically possible combinations.
- The database relation contains only the tuples that actually belong to the relation.
- In simple terms:
- Cartesian Product $\rightarrow$ All Possible Combinations
- Relation $\rightarrow$ Relevant/Valid Tuples
- $ID \times Name \times Age \rightarrow$ Possible Tuples $\rightarrow$ Student Relation



Core Terminology

Relational Term	Meaning	Example
Relation	Table	Student
Tuple	Row/record	(101, Rahul, CSE)
Attribute	Column/property	Name
Domain	Set of permitted values	CSE, ECE, ME
Degree	Number of attributes	3
Cardinality	Number of tuples	2

- **Memory Aid**
- **Relation → Table**
- **Tuple → Row**
- **Attribute → Column**
- **Domain → Allowed Values**



Relation, Tuple, Attribute & Domain

Relation

- A named collection of tuples representing an entity or relationship.

Tuple

- A single row/record in a relation.

Example:

- (101, Rahul, CSE)

Attribute

- A named property describing an entity.

Examples:

- RollNo
- Name
- Department

Domain

- The set of valid values an attribute may contain.

Examples:

- Department → {CSE, ECE, ME, CE}
- Age → 18-100
- Salary → Positive numeric values

Important:

- Note that a domain includes the idea of what values are meaningful, not merely the underlying data type.
- For example, Age may be an integer data type, but a sensible domain might restrict it to a meaningful range.



Properties of a Relation

A valid relation follows important principles:

- Each relation has a unique name.
- Attribute names within a relation are unique.
- Attribute values are atomic.
- Values of an attribute come from the same domain.
- Duplicate tuples are not allowed in the pure relational model.
- Row order is not significant.
- Column order is not significant.

Atomic Values Explained:

RollNo	Name	Subjects
101	Rahul	DBMS, AI

Better relational design

- Separate subjects into related records/tables.

When we violate this principle, searching and updating become difficult. Later, normalization gives us systematic techniques for fixing such designs



Relation Schema vs. Relation Instance

- Relation Schema = Structure
- Example:
- STUDENT (RollNo, Name, Department, Email)
- It describes:
- Relation name
- Attributes
- Domains/data types
- Constraints
- Relation Instance = Actual Data

RollNo	Name	Department
101	Rahul	CSE
102	Anita	ECE

- Easy Analogy
- Schema = Blueprint
- Instance = Current Contents
- The schema defines what the relation looks like, while the instance represents the data stored at a particular point in time.
- An INSERT, UPDATE, or DELETE can change the instance without necessarily changing the schema.



Database Schema: The Bigger Picture

Database Schema

- The database schema describes the overall logical structure of the database, including its relations and their relationships/constraints.

Example:

- Student (RollNo, Name, DeptID)
- Department (DeptID, DepartmentName)
- Course (CourseID, Title, Credits)
- Faculty (FacultyID, Name, DeptID)
- Enrollment (RollNo, CourseID)
- Relation Schema vs Database Schema

Relation Schema	Database Schema
Describes one relation	Describes the entire database
Local structure	Global structure
One table's design	Collection of related table designs

- Analogy:
- Relation schema = blueprint of one building
- Database schema = blueprint of the entire campus



Keys and mathematical dependencies

The fundamental question:

- How can we uniquely identify a particular tuple?

Keys help us:

- Recognize tuples uniquely
- Avoid ambiguity
- Establish relationships
- Maintain integrity
- Support efficient data access
- Example: RollNo uniquely identifies a student

Definition

- A key is one or more attributes that uniquely identify a tuple in a relation.



Types of Keys

Super Key

- Any attribute or combination of attributes that uniquely identifies a tuple.
- Examples: {RollNo} {RollNo, Name}

Candidate Key

- A minimal super key.
- Example: {RollNo} {Email} if Email is unique

Primary Key

- The candidate key selected as the principal identifier.
- Example: RollNo

Alternate Key

- Candidate key(s) not selected as the primary key.
- Example: Email

Key Hierarchy

- Super Key → Candidate Keys → Primary Key / Alternate Key
- If RollNo alone uniquely identifies a student, then (RollNo, Name) is a super key but not a candidate key because Name is unnecessary.



Composite Keys & Foreign Keys

Composite Key

- A key consisting of two or more attributes.

- Example:

Enrollment

RollNo CourseID

101 CS101

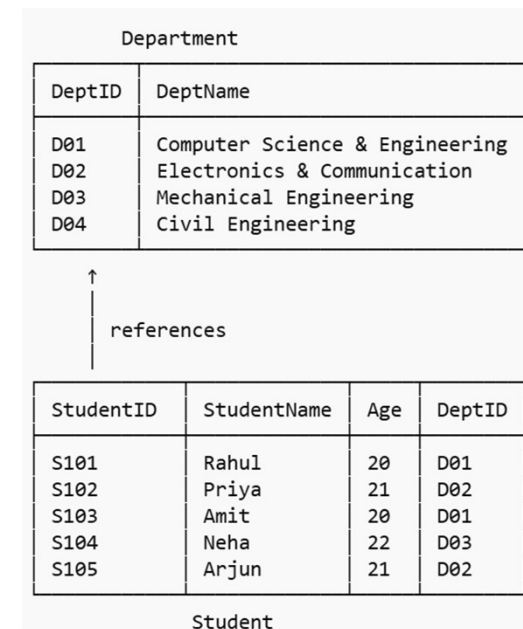
101 CS102

102 CS101

- Primary Key: (RollNo, CourseID)
- The combination identifies each enrollment uniquely.

Foreign Key

- An attribute that references a key in another relation.
- Student.DeptID → Department.DeptID
- Purpose
- Connect tables
- Represent relationships
- Support referential integrity



Functional Dependencies: The Mathematics Behind Keys

- A Functional Dependency expresses determination.
- Suppose: `Student(StudentID, StudentName, Age)`
- We can write: `StudentID → StudentName, Age`
- Meaning: If two tuples have the same `StudentID`, they must have the same `StudentName` and `Age`.
- Why does this matter?
 - Functional dependencies help us reason about:
 - Candidate keys
 - Primary keys
 - Redundancy
 - Normalization
 - 1NF, 2NF, 3NF
 - BCNF
- Example
 - `StudentID → StudentName`
 - `CourseID → CourseName, Instructor`
 - `(StudentID, CourseID) → Grade`



Normalization: Mathematics Applied to Database Design

Consider:

• StudentID StudentName CourseID CourseName Instructor

We may have dependencies:

- StudentID $\rightarrow$ StudentName
- CourseID $\rightarrow$ CourseName, Instructor

Keeping everything in one table causes repeated information.

- Better decomposition

Student

- (StudentID, StudentName)

Course

- (CourseID, CourseName, Instructor)

Enrollment

- (StudentID, CourseID)

Core Idea

- Functional Dependencies $\rightarrow$ Identify Redundancy $\rightarrow$ Decompose Relations



NULL Values & Integrity

NULL Values

- NULL indicates information that is:
- Unknown
- Missing
- Not applicable

NULL does not mean:

- Zero
- Empty string
- False

Integrity Constraints

Entity Integrity

Primary key values cannot be NULL.

Referential Integrity

A foreign key value must correspond to an existing referenced key, subject to the defined constraints.

Mathematical Perspective

Conceptually:

Enrollment.StudentID $\subseteq$ Student.StudentID

Meaning:

Every student ID appearing in Enrollment must also appear in Student.

Domain Integrity

Values must satisfy their defined domain and constraints.



SQL and Relational Algebra

Relational Algebra = mathematical foundation of SQL

Selection → WHERE

Relational algebra: σ

SQL: SELECT ... WHERE ...

Projection → choosing columns

Relational algebra: π

SQL: SELECT column1, column2

Join → combining relations

Relational algebra: $\bowtie$

SQL: JOIN

Union → combining compatible results

Relational algebra: $\cup$

SQL: UNION

Key idea: SQL provides a practical language for operations that have a formal foundation in relational algebra.



Why the Relational Model Still Matters in the NoSQL Era

Strong structure: Tables, keys, and constraints provide clear data organization.

Relationships: Relational databases naturally represent connections between data using foreign keys and joins.

Consistency: Transactions and integrity constraints help maintain reliable data.

Powerful querying: SQL supports complex queries and data analysis.

Foundation for database theory: Concepts such as keys, functional dependencies, normalization, and relational algebra remain fundamental.

NoSQL is complementary: NoSQL databases are useful for specific needs such as massive scale, flexible schemas, or specialized data models—not necessarily a replacement for relational databases.



Summary

- We learned
- ✓ Relational Model
- ✓ Relation, Tuple & Attribute
- ✓ Domain, Degree & Cardinality
- ✓ Relation Schema & Database Schema
- ✓ Super Keys & Candidate Keys
- ✓ Primary, Alternate & Composite Keys
- ✓ Foreign Keys
- ✓ Functional Dependencies
- ✓ Normalization
- ✓ NULL Values
- ✓ Integrity Constraints
- ✓ Referential Integrity
- ✓ Relational Algebra & SQL
- ✓ Relational Model vs. NoSQL



haribansh@bhu.ac.in

Learning Reflections

1. What is a relation in the Relational Model? Explain the difference between a relation, tuple, and attribute.
2. What are super key, candidate key, primary key, alternate key, composite key, and foreign key? How are they different from one another?
3. What is a functional dependency? Why is it important for normalization? Explain with a simple example.
4. What is NULL in a relational database? How is it different from zero or an empty string? Also, what are the major integrity constraints in a relational database?
5. What is referential integrity? How do SQL operations relate to relational algebra, and why is the Relational Model still important even with the rise of NoSQL databases?



Homework

- Explain the concepts of relation, tuple, attribute, domain, degree, cardinality, relation schema, relation instance, and database schema in detail. Discuss the concept of keys and elaborate their importance.





haribansh@bhu.ac.in

Thanks!

📌 Heads Up!

These slides are just a **guide** — not the full story!

To really learn, make sure to read your book, and explore other resources too.